LAPORAN PRAKTIKUM
STRUKTUR DATA DAN ALGORITMA

Disusun Oleh:

SAFA AURA

NIM	**: 2025573010024**
KELAS	**: TI-1D**
JURUSAN	**: Teknologi Informasi dan Komputer**
PRODI	**: Teknik Informatika**
DOSEN PENGAJAR	**: M. Reza Zulman, SST. M.Sc**

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulisan materi *Struktur Data dan Algoritma Menggunakan JavaScript & Node.js* dengan fokus pada *Function, Arrow Function, Callback, serta Array dan Method Array* ini dapat diselesaikan dengan baik.

Materi ini disusun dengan tujuan untuk memberikan pemahaman dasar hingga menengah mengenai konsep penting dalam pemrograman, khususnya dalam pengolahan data dan penyusunan logika algoritma menggunakan JavaScript. Dalam dunia pengembangan perangkat lunak modern, pemahaman terhadap struktur data dan algoritma menjadi sangat penting karena berperan besar dalam meningkatkan efisiensi dan performa aplikasi.

Pembahasan dalam materi ini mencakup penggunaan *Function* sebagai dasar modularisasi kode, *Arrow Function* yang menawarkan sintaks lebih ringkas, serta *Callback* yang menjadi fondasi dalam pemrograman asinkron di JavaScript, terutama dalam lingkungan Node.js. Selain itu, konsep *Array* beserta berbagai *method*-nya juga dibahas untuk membantu dalam pengelolaan dan manipulasi data secara efektif.

Penulis menyadari bahwa materi ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan di masa mendatang. Semoga materi ini dapat memberikan manfaat dan membantu pembaca dalam memahami serta mengimplementasikan konsep struktur data dan algoritma menggunakan JavaScript dengan lebih baik.

Akhir kata, semoga karya ini dapat menjadi referensi yang bermanfaat bagi para pembaca, khususnya bagi yang sedang mempelajari pemrograman JavaScript dan Node.js.

Buketrata,…… 2026

Safa Aura
2025573010024

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Perumusan Masalah 1
1.3 Tujuan Penulisan 2
1.4 Manfaat Penulisan 2
BAB II DASAT TEORI 3
2.1 Function (fungsi) 3
2.2 Arrow Function 3
2.3 Callback Function 3
2.4 Array 4
2.5 Method Array 4
BAB III MTODE PENULISAN 5
31 Alat dan Bahan 5
3.2 Langkah- Langkah Praktikum Function dan Array 5
3.3 Hasil Praktikum 11
3.4 Summit ke Github 12
3.5 Analisis Hasil 13
BAB VI PENUTUPAN 14
4.1 Kesimpulan 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi yang semakin pesat di era digital saat ini menuntut setiap individu, khususnya di bidang teknologi, untuk memiliki kemampuan pemrograman yang baik. Tidak hanya sekadar mampu menulis kode, seorang programmer juga dituntut untuk mampu menyusun logika pemrograman yang sistematis, efisien, dan mudah dipahami. Kemampuan ini sangat penting agar program yang dibuat tidak hanya berjalan dengan benar, tetapi juga memiliki performa yang optimal serta mudah untuk dikembangkan di masa mendatang.

Dalam konteks tersebut, pemahaman mengenai struktur data dan algoritma menjadi salah satu aspek fundamental yang harus dikuasai. Struktur data berperan dalam mengatur dan menyimpan data secara terorganisir, sedangkan algoritma digunakan untuk menyelesaikan suatu permasalahan melalui langkah-langkah yang logis dan terstruktur. Keduanya saling berkaitan dan sangat berpengaruh terhadap efisiensi serta efektivitas suatu program.

Salah satu bahasa pemrograman yang широко digunakan saat ini adalah JavaScript. JavaScript tidak hanya digunakan untuk pengembangan sisi klien (client-side), tetapi juga dapat digunakan di sisi server (server-side) dengan bantuan teknologi seperti Node.js. Hal ini menjadikan JavaScript sebagai bahasa yang fleksibel dan banyak diminati dalam pengembangan aplikasi modern, baik berbasis web maupun sistem lainnya.

Dalam mempelajari JavaScript, terdapat beberapa konsep dasar yang sangat penting untuk dipahami, seperti function, arrow function, callback, serta array beserta berbagai method yang dimilikinya. Function digunakan untuk mengelompokkan kode agar lebih terstruktur dan dapat digunakan kembali. Arrow function memberikan alternatif penulisan fungsi yang lebih ringkas dan modern. Callback memungkinkan pengaturan alur eksekusi program, terutama dalam proses asynchronous. Sementara itu, array dan method array berperan penting dalam pengolahan data secara efisien.

Pemahaman terhadap konsep-konsep tersebut akan membantu dalam meningkatkan kemampuan berpikir logis dan analitis dalam pemrograman. Selain itu, penggunaan konsep yang tepat juga dapat menghasilkan kode yang lebih bersih (clean code), mudah dibaca, serta lebih mudah untuk dipelihara dan dikembangkan.

Dengan demikian, materi ini disusun sebagai dasar pembelajaran dalam memahami struktur data dan algoritma menggunakan JavaScript dan Node.js. Diharapkan melalui pembahasan ini, pembaca dapat memiliki pemahaman yang lebih baik serta mampu mengaplikasikan konsep-konsep tersebut dalam pembuatan program yang lebih optimal, efisien, dan sesuai dengan kebutuhan perkembangan teknologi saat ini.

1.2 Perumusan Masalah

1. Bagaimana memahami dan menggunakan *function* serta *arrow function* dalam JavaScript?
2. Bagaimana penerapan *callback* dalam pemrograman JavaScript, khususnya pada Node.js?

3. Bagaimana penggunaan *array* dan berbagai *method*-nya dalam mengelola dan memproses data?

1.3 Tujuan Penulisan

1. Untuk memahami konsep dan penggunaan *function* serta *arrow function* dalam JavaScript.
2. Untuk mengetahui cara penerapan *callback* dalam pemrograman JavaScript, khususnya pada Node.js.
3. Untuk memahami penggunaan *array* dan berbagai *method*-nya dalam mengelola dan memproses data secara efektif.

1.4 Manfaat Penulisan

1. Memberikan pemahaman yang lebih baik mengenai *function*, *arrow function*, dan *callback* dalam JavaScript.
2. Membantu pembaca memahami konsep *array* dan *method array* untuk pengelolaan data yang lebih efisien.
3. Menjadi referensi praktis bagi pembaca dalam mengembangkan program menggunakan JavaScript dan Node.js dengan struktur kode yang rapi dan efektif.

BAB II
DASAT TEORI

2.1 Function (fungsi)

Function atau fungsi merupakan salah satu konsep dasar dalam pemrograman yang digunakan untuk mengelompokkan sekumpulan instruksi atau perintah ke dalam satu kesatuan yang memiliki tujuan tertentu. Dengan adanya function, program dapat disusun secara lebih terstruktur dan modular, sehingga memudahkan dalam proses pengembangan, pengujian, maupun pemeliharaan kode. Setiap function umumnya dirancang untuk menyelesaikan satu tugas spesifik, sehingga prinsip pemrograman yang baik seperti modularitas dan reusabilitas dapat tercapai.

Penggunaan function juga memungkinkan pengurangan duplikasi kode (code redundancy), karena suatu fungsi yang telah dibuat dapat digunakan kembali di berbagai bagian program tanpa harus menuliskan ulang kode yang sama. Selain itu, function dapat menerima parameter sebagai input yang bersifat dinamis, sehingga fungsi tersebut dapat digunakan dalam berbagai kondisi yang berbeda. Function juga dapat mengembalikan nilai (return value) sebagai hasil dari proses yang dilakukan, sehingga dapat digunakan kembali dalam proses perhitungan atau pengolahan data lainnya.

Dalam konteks yang lebih luas, function berperan penting dalam membangun aplikasi yang kompleks, karena membantu membagi sistem menjadi bagian-bagian kecil yang lebih mudah dikelola. Dengan demikian, pemahaman yang baik mengenai function akan sangat membantu dalam meningkatkan kualitas dan efisiensi program yang dikembangkan.

2.2 Arrow Function

Arrow function merupakan bentuk penulisan fungsi yang lebih modern dan ringkas yang diperkenalkan dalam standar ECMAScript 6 (ES6). Arrow function dirancang untuk menyederhanakan sintaks penulisan fungsi, sehingga programmer dapat menuliskan fungsi dengan lebih singkat tanpa mengurangi fungsionalitasnya. Hal ini sangat membantu dalam meningkatkan keterbacaan kode, terutama pada program yang memiliki banyak fungsi sederhana.

Salah satu karakteristik utama dari arrow function adalah tidak memiliki binding this sendiri, melainkan mewarisi nilai this dari lingkungan di mana fungsi tersebut didefinisikan (lexical scoping). Perbedaan ini menjadi penting dalam konteks tertentu, terutama saat bekerja dengan objek atau event handler. Dengan sifat tersebut, arrow function sering digunakan dalam situasi yang membutuhkan konsistensi nilai this, seperti dalam penggunaan callback atau method array.

Selain itu, arrow function juga mendukung penulisan yang lebih minimalis, misalnya dengan menghilangkan tanda kurung pada parameter tunggal atau tanpa menggunakan kata kunci return pada ekspresi sederhana. Dengan berbagai keunggulan tersebut, arrow function menjadi salah satu fitur penting dalam JavaScript modern yang banyak digunakan untuk meningkatkan efisiensi dan kejelasan kode.

2.3 Callback Function

Callback function adalah fungsi yang dijadikan sebagai argumen dalam fungsi lain untuk kemudian dipanggil kembali setelah suatu proses selesai dilakukan. Konsep ini memungkinkan pengaturan alur eksekusi program menjadi lebih fleksibel, terutama dalam menangani proses asynchronous seperti pengambilan data atau penundaan waktu.

Dengan menggunakan callback, program dapat tetap berjalan tanpa harus menunggu proses tertentu selesai terlebih dahulu. Namun, penggunaan callback yang berlebihan dapat menyebabkan kode menjadi sulit dibaca, sehingga perlu digunakan secara bijak agar tetap terstruktur dan mudah dipahami.

2.4 Array

Array merupakan salah satu struktur data dasar yang digunakan untuk menyimpan sekumpulan nilai dalam satu variabel. Data yang disimpan dalam array tersusun secara berurutan dan masing-masing elemen dapat diakses menggunakan indeks, yang dimulai dari nol. Array memungkinkan penyimpanan berbagai jenis data, baik yang bersifat homogen (jenis data sama) maupun heterogen (berbeda jenis data), tergantung pada kebutuhan program.

Keberadaan array sangat penting dalam pemrograman karena memberikan cara yang efisien dalam mengelola data dalam jumlah besar. Tanpa array, programmer harus membuat banyak variabel terpisah untuk menyimpan data, yang tentunya akan menyulitkan dalam pengelolaan dan pengolahan data tersebut. Dengan menggunakan array, data dapat dengan mudah ditambahkan, diubah, dihapus, maupun diakses sesuai kebutuhan.

Selain itu, array juga menjadi dasar dalam berbagai struktur data yang lebih kompleks, seperti stack, queue, dan list. Oleh karena itu, pemahaman mengenai array tidak hanya penting untuk penggunaan dasar, tetapi juga menjadi fondasi dalam mempelajari konsep struktur data yang lebih lanjut. Dengan pengelolaan array yang baik, program dapat berjalan lebih efisien dan terorganisir.

2.5 Method Array

Method array adalah fungsi bawaan dalam JavaScript yang digunakan untuk melakukan berbagai operasi pada array, seperti penyaringan, pengubahan, pencarian, dan pengulangan data. Method ini mempermudah proses pengolahan data tanpa perlu membuat algoritma secara manual.

Penggunaan method array mendukung penulisan kode yang lebih ringkas, mudah dibaca, dan efisien. Selain itu, method array juga membantu mengurangi kesalahan dalam pengolahan data, sehingga programmer dapat lebih fokus pada logika program. Oleh karena itu, penguasaan method array sangat penting dalam pengembangan aplikasi JavaScript.

BAB III
MTODE PENULISAN

31 Alat dan Bahan

Alat

1. Komputer atau laptop
2. Mouse dan keyboard
3. Akses internet

Bahan

1. Git Bash
2. Visual Studio Code

3.2 Langkah- Langkah Praktikum Function dan Array

1. Buka aplikasi Gitt Bash.
2. Pertama-tama ketik Bash: cd /d
3. Kemudian ketik : cd "/d/Nama Folder yang Sudah Di Cloning"
4. cd nama Ripository
5. mkdir praktikum-3
6. cd praktikum-3

```
HP@LAPTOP-08FUNPL7 MINGW64 ~
$ cd /d

HP@LAPTOP-08FUNPL7 MINGW64 /d
$ cd "/d/praktikum struktur data & algoritma"

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma
$ cd 225573010024_strukturdataalgoritma

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma (main)
$ mkdir praktikum-03

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalg
oritma (main)
$ mkdir praktikum-3

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalg
oritma (main)
$ cd praktikum-3
```

7. npm init -y

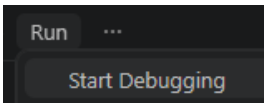
```
HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalg
oritma/praktikum-3 (main)
$ npm init -y
Wrote to D:\praktikum struktur data & algoritma\225573010024_strukturdataalgoritma\praktikum-3
\package.json:

{
  "name": "praktikum-3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

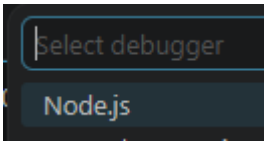
8. kemudian buka VS Code
9. ketik : touch 01-function-dasar.js pada Git Bash
10. Masukkan coding ke aplikasi VS Vode

```
JS 01-function-dasar.js > ...
1   function sapa() {
2       console.log('Halo! Selamat datang di praktikum JavaScript.');
3   }
4   sapa();
5   sapa();
6   function sapaNama(nama) {
7       console.log(`Halo, ${nama}! Selamat belajar.`);
8   }
9   sapaNama('Budi');
10  sapaNama('Siti');
11  function tambah(angka1, angka2) {
12      const hasil = angka1 + angka2;
13      return hasil;
14  }
15  const hasilPenjumlahan = tambah(10, 25);
16  console.log('10 + 25 =', hasilPenjumlahan);
17  console.log('7 + 13 =', tambah(7, 13));
18  function hitung(nilai, pengali = 2) {
19      return nilai * pengali;
20  }
21  console.log(hitung(5));
22  console.log(hitung(5, 3));
23  const pesanGlobal = 'Saya ada di mana saja';
24
25  function cekScope() {
26      const pesanLokal = 'Saya hanya ada di dalam function ini';
27      console.log(pesanGlobal);
28      console.log(pesanLokal);
29  }
30  cekScope();
31  console.log(pesanGlobal);
```

11. Run lalu tekan Start Debugging



12. Lalu pilih Node.js



13. Maka akan keluar outputnya

```
C:\Program Files\nodejs\node.exe .\01-function-dasar.js
(2) Halo! Selamat datang di praktikum JavaScript.        function-dasar.js:2-01
Halo, Budi! Selamat belajar.                             function-dasar.js:7-01
Halo, Siti! Selamat belajar.                             function-dasar.js:7-01
10 + 25 = 35                                             function-dasar.js:16-01
7 + 13 = 20                                              function-dasar.js:17-01
10                                                       function-dasar.js:21-01
15                                                       function-dasar.js:22-01
Saya ada di mana saja                                    function-dasar.js:27-01
Saya hanya ada di dalam function ini                     function-dasar.js:28-01
Saya ada di mana saja                                    function-dasar.js:31-01
```

14. ketik: node 01-function-dasar.js pada Git Bash, maka akan keluar hasil yang seperti ini.

```
HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalg
oritma/praktikum-3 (main)
$ touch 01-function-dasar.js

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalg
oritma/praktikum-3 (main)
$ node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
```

15. Lanjutkan ke bagian berikutnya, ulangi seperti Langkah-langkas diatas

16. Ketik touch 02-arrow-callback.js pada Git Bash.

17. Masukkan coding dan tampilkan outputnya.

```js
JS 02-arrow-callback.js > lakukanOperasi
function kuadratBiasa(x) {
    return x * x;
}
const kuadrat = (x) => {
    return x * x;
};
const kuadratRingkas = x => x * x;
console.log('=== Perbandingan Penulisan ===');
console.log('Biasa :', kuadratBiasa(5));
console.log('Arrow :', kuadrat(5));
console.log('Ringkas :', kuadratRingkas(5));

const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;

console.log('Luas 4x6 :', luas(4, 6));
console.log(salam('Budi', 'Pagi'));

function lakukanOperasi(angka, operasiCallback) {
    console.log(`Angka awal: ${angka}`);
    const hasil = operasiCallback(angka);
    console.log(`Hasil setelah operasi: ${hasil}`);
}
console.log('\n=== Callback ===');
```

```
C:\Program Files\nodejs\node.exe .\02-arrow-callback.js
=== Perbandingan Penulisan ===                    arrow-callback.js:8-02
Biasa : 25                                        arrow-callback.js:9-02
Arrow : 25                                        arrow-callback.js:10-02
Ringkas : 25                                      arrow-callback.js:11-02
Luas 4x6 : 24                                     arrow-callback.js:16-02
Selamat Pagi, Budi!                               arrow-callback.js:17-02
                                                  arrow-callback.js:24-02
=== Callback ===                                  arrow-callback.js:24-02
```

18.ketik: node 02-arrow-callback.js pada Git Bash.

```
HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalgoritma/praktikum-3 (main)
$ touch 02-arrow-callback.js

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_strukturdataalgoritma/praktikum-3 (main)
$ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
```

19. Ketik touch 03-array-dasar.js pada Git Bash.

20. Masukkan coding dan tampilkan outputnya.

```
JS 03-array-dasar.js > ...
const mahasiswa = ['Budi', 'Siti', 'Ahmad', 'Rina'];
const nilai = [85, 92, 78, 95, 88];

console.log('=== Array Awal ===');
console.log('Mahasiswa:', mahasiswa);
console.log('Nilai :', nilai);
console.log('Jumlah mahasiswa:', mahasiswa.length);

console.log('\n=== Akses Elemen ===');
console.log('Elemen pertama :', mahasiswa[0]); // indeks 0
console.log('Elemen ketiga :', mahasiswa[2]); // indeks 2
console.log('Elemen terakhir:', mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = 'Siti Rahayu';
console.log('\nSetelah diubah:', mahasiswa);

console.log('\n=== Manipulasi Array ===');
mahasiswa.push('Doni');
console.log('Setelah push :', mahasiswa);
mahasiswa.unshift('Zahra');
console.log('Setelah unshift :', mahasiswa);
const dihapusAkhir = mahasiswa.pop();
console.log('Dihapus (pop) :', dihapusAkhir);
console.log('Setelah pop :', mahasiswa);
const dihapusAwal = mahasiswa.shift();
console.log('Dihapus (shift) :', dihapusAwal);
console.log('Setelah shift :', mahasiswa);

console.log('\n=== Pencarian ===');
console.log('Indeks Ahmad :', mahasiswa.indexOf('Ahmad'));
console.log('Ada Rina? :', mahasiswa.includes('Rina'));
console.log('Ada Budi? :', mahasiswa.includes('Budi'));

const sebagian = nilai.slice(1, 4);
console.log('\nNilai asli :', nilai);
console.log('Slice [1,4] :', sebagian);
```

```
C:\Program Files\nodejs\node.exe .\03-array-dasar.js
=== Array Awal ===
> Mahasiswa: (4) ['Budi', 'Siti', 'Ahmad', 'Rina']
> Nilai : (5) [85, 92, 78, 95, 88]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

> Setelah diubah: (4) ['Budi', 'Siti Rahayu', 'Ahmad', 'Rina']

=== Manipulasi Array ===
> Setelah push : (5) ['Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni']
> Setelah unshift : (6) ['Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni']
Dihapus (pop) : Doni
> Setelah pop : (5) ['Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina']
Dihapus (shift) : Zahra
> Setelah shift : (4) ['Budi', 'Siti Rahayu', 'Ahmad', 'Rina']

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

> Nilai asli : (5) [85, 92, 78, 95, 88]
> Slice [1,4] : (3) [92, 78, 95]
```

21. Ketik node 03-array-dasar.js pada Git Bash.

```
HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma/praktikum-3 (main)
$ touch 03-array-dasar.js

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma/praktikum-3 (main)
$ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]
```

22. Ketik tpuch 04-array-method.js pada Git Bash.

23. Masukkan coding dan tampilkan outputnya.

```
JS 04-array-method.js > gradeHuruf > nilaiMahasiswa.map() callback
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
console.log('=== forEach: Tampilkan Semua Nilai ===');
nilaiMahasiswa.forEach((nilai, indeks) => {
console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});
const gradeHuruf = nilaiMahasiswa.map(nilai => {
    if (nilai >= 90) return 'A';
    if (nilai >= 80) return 'B';
    if (nilai >= 70) return 'C';
    if (nilai >= 60) return 'D';
    return 'E';
});
console.log('\n=== map: Nilai ke Grade ===');
console.log('Nilai :', nilaiMahasiswa);
console.log('Grade :', gradeHuruf);

const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);

console.log('\n=== filter: Lulus dan Tidak Lulus ===');
console.log('Semua nilai :', nilaiMahasiswa);
console.log('Nilai lulus :', nilaiLulus);
console.log('Nilai gagal :', nilaiGagal);
```

```
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
    return akumulator + nilai;
}, 0);

const rataRataNilaiLulus = nilaiMahasiswa
    .filter(nilai => nilai >= 60)
    .reduce((acc, val, idx, arr) => {
        return acc + val / arr.length;
}, 0);

console.log('\n=== Method Chaining ===');
console.log('Rata-rata nilai lulus:', rataRataNilaiLulus.toFixed(2));
```

```
C:\Program Files\nodejs\node.exe .\04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
> Nilai : (8) [75, 90, 55, 82, 68, 95, 48, 77]
> Grade : (8) ['C', 'A', 'E', 'B', 'D', 'A', 'E', 'C']

=== filter: Lulus dan Tidak Lulus ===
> Semua nilai : (8) [75, 90, 55, 82, 68, 95, 48, 77]
> Nilai lulus : (6) [75, 90, 82, 68, 95, 77]
> Nilai gagal : (2) [55, 48]

=== Method Chaining ===
Rata-rata nilai lulus: 81.17
```

24 Ketik node 04-array-method.js pada Git Bash.

```
HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma/praktikum-3 (main)
$ touch 04-array-method.js

HP@LAPTOP-08FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma/praktikum-3 (main)
$ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]

=== Method Chaining ===
Rata-rata nilai lulus: 81.17
```

25. Ketik touch 05-rekursi pada Git Bash.

26. Masukkan coding dan tampilkan outputnya.

```
JS 05-rekursi.js > faktorial
function faktorial(n) {
// Base case: berhenti saat n = 1
if (n <= 1) return 1;
    // Recursive case: n × faktorial(n-1)
    return n * faktorial(n - 1);
}
console.log('=== Faktorial ===');
console.log('0! =', faktorial(0));
console.log('1! =', faktorial(1));
console.log('5! =', faktorial(5));
console.log('7! =', faktorial(7));

function fibonacci(n) {
    // Base case
    if (n === 0) return 0;
    if (n === 1) return 1;
    // Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log('\n=== Fibonacci ===');
for (let i = 0; i <= 8; i++) {
    process.stdout.write(fibonacci(i) + ' ');
}
console.log(''); // pindah baris
```

```javascript
function jumlahArray(arr, indeks = 0) {
    // Base case: sudah sampai akhir array
    if (indeks === arr.length) return 0;

    // Recursive case: elemen saat ini + jumlah sisa array
    return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log('\n=== Jumlah Array dengan Rekursi ===');
console.log('Array :', angka);
console.log('Jumlah :', jumlahArray(angka));

function countdown(n) {
    if (n < 0) {
        console.log('Selesai!');
        return;
}
    console.log(n);
    countdown(n - 1);
}
console.log('\n=== Countdown Rekursif ===');
countdown(5);
```

```
C:\Program Files\nodejs\node.exe .\05-rekursi.js
=== Faktorial ===                                   rekursi.js:7-05
0! = 1                                              rekursi.js:8-05
1! = 1                                              rekursi.js:9-05
5! = 120                                            rekursi.js:10-05
7! = 5040                                           rekursi.js:11-05
                                                    rekursi.js:20-05
=== Fibonacci ===                                   rekursi.js:20-05
                                                    rekursi.js:24-05
                                                    rekursi.js:34-05
=== Jumlah Array dengan Rekursi ===                 rekursi.js:34-05
> Array : (5) [3, 7, 2, 9, 5]                       rekursi.js:35-05
Jumlah : 26                                         rekursi.js:36-05
                                                    rekursi.js:46-05
=== Countdown Rekursif ===                          rekursi.js:46-05
5                                                   rekursi.js:43-05
4                                                   rekursi.js:43-05
3                                                   rekursi.js:43-05
2                                                   rekursi.js:43-05
1                                                   rekursi.js:43-05
0                                                   rekursi.js:43-05
Selesai!                                            rekursi.js:40-05
```

27. Ketik node 05-rekursi pada Git Bash.

```
HP@LAPTOP-O8FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma/praktikum-3 (main)
$ touch 05-rekursi.js

HP@LAPTOP-O8FUNPL7 MINGW64 /d/praktikum struktur data & algoritma/225573010024_s
trukturdataalgoritma/praktikum-3 (main)
$ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
```

3.3 Hasil Praktikum

1. output 01-function-dasar.js

```
5 + 3 = 30                                          function-dasar.js:65-01
10 - 7 = 13                                         function-dasar.js:66-01
4 * 6 = 24                                          function-dasar.js:67-01
20 / 4 = 5.5                                        function-dasar.js:68-01
Error: tidak bisa membagi dengan nol                function-dasar.js:44-01
10 / 0 = null                                       function-dasar.js:69-01
```

2. Output 02-arrow-callback.js

```
=== proses kalimat ===                              arrow-callback.js:35-02
BELAJAR JAVASCRIPT ITU MENYENANGKAN                 arrow-callback.js:33-02
belajar javascript itu menyenangkan!                arrow-callback.js:33-02
4                                                   arrow-callback.js:33-02
```

3.Output 03-array-dasar.js

```
=== Daftar Belanja Awal ===                         array-dasar.js:31-03
1. Beras                                            array-dasar.js:33-03
2. Gula                                             array-dasar.js:33-03
3. Telur                                            array-dasar.js:33-03
4. Kopi                                             array-dasar.js:33-03
                                                    array-dasar.js:37-03
Setelah tambah item:                                array-dasar.js:37-03
1. Beras                                            array-dasar.js:39-03
2. Gula                                             array-dasar.js:39-03
3. Telur                                            array-dasar.js:39-03
4. Kopi                                             array-dasar.js:39-03
5. Susu                                             array-dasar.js:39-03
6. Roti                                             array-dasar.js:39-03
                                                    array-dasar.js:42-03
Item yang dihapus: Beras                            array-dasar.js:42-03
Apakah "Susu" ada di daftar? true                   array-dasar.js:44-03
Jumlah item akhir: 5                                array-dasar.js:45-03
```

4. Ouput 04-array-method

```
  Nama produk yang tersedia:                        array-method.js:52-04
> (3) ['Laptop', 'Keyboard', 'Headset']             array-method.js:53-04
                                                    array-method.js:57-04
  Total nilai inventaris produk tersedia:           array-method.js:57-04
  Rp99.300.000                                      array-method.js:58-04
                                                    array-method.js:59-04
  Daftar semua produk:                              array-method.js:59-04
  [TERSEDIA] Laptop - Rp8.500.000 (5 unit)          array-method.js:63-04
  [HABIS] Mouse - Rp150.000 (0 unit)                array-method.js:63-04
  [TERSEDIA] Keyboard - Rp4.500.000 (12 unit)       array-method.js:63-04
  [HABIS] Monitor - Rp3.200.000 (0 unit)            array-method.js:63-04
  [TERSEDIA] Headset - Rp350.000 (8 unit)           array-method.js:63-04
```

3.4 Summit ke Github

1. masuk git bash, lalu ketik :
2. Cd /d
3. Masuk ke folder yang sudah disiapkan sebelumnya Cd “/d/praktikum struktur data & algoritma”
4. Masuk ke repository : cd 225573010024_strukturdataalgoritma
5. Kemudian masuk ke folder praktikum-3: cd praktikum-3
6. Kembali ke repository : cd ..

7. Ketik : git add praktikum-3/
8. Ketik git commit -m “Praktikum 3: Function & Array – arrow function, callback, iterasi array, rekursi”
9. Masukan email dan user name github kamu.
10. Setelah mengisi email dan user ke gitbub, akan otomatis teralihkan. Kemudian, isi username, email, beserta password github.
11. Ketik :git push origin main.
12. Tunggu proses summit, masukan email, atau username github
13. buka github di browser, dan buka bagian repository, selesai

3.5 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, dapat dianalisis bahwa penggunaan function dalam JavaScript sangat membantu dalam menyusun program agar lebih terstruktur dan mudah dipahami. Function memungkinkan pengelompokan kode menjadi bagian-bagian kecil yang memiliki fungsi tertentu sehingga dapat digunakan kembali tanpa harus menulis kode yang sama secara berulang.

Pada praktikum kedua, penggunaan arrow function menunjukkan bahwa JavaScript modern menyediakan cara penulisan fungsi yang lebih sederhana dan ringkas. Arrow function mempermudah penulisan kode terutama pada fungsi yang sederhana serta sering digunakan pada proses callback.

Selanjutnya, pada penerapan callback function dapat dilihat bahwa fungsi dapat dipanggil kembali setelah suatu proses selesai dijalankan. Hal ini sangat berguna dalam pengaturan alur program, khususnya pada proses yang bersifat asynchronous. Dengan menggunakan callback, program dapat berjalan secara lebih fleksibel tanpa harus menunggu proses sebelumnya selesai secara langsung.

Pada praktikum array dasar, terlihat bahwa array sangat efektif digunakan untuk menyimpan dan mengelola sekumpulan data dalam satu variabel. Data dalam array dapat diakses melalui indeks sehingga mempermudah proses pengolahan data dalam jumlah banyak.

Kemudian pada praktikum method array, berbagai method seperti pengulangan dan manipulasi data dapat digunakan untuk memproses isi array dengan lebih efisien. Penggunaan method array membuat kode program menjadi lebih singkat, jelas, dan mudah dibaca dibandingkan dengan menggunakan perulangan manual.

Secara keseluruhan, hasil praktikum menunjukkan bahwa pemahaman terhadap function, arrow function, callback, serta array dan method array sangat penting dalam pengembangan program JavaScript. Konsep-konsep tersebut membantu programmer dalam menulis kode yang lebih terstruktur, efisien, dan mudah dikembangkan.

BAB VI
PENUTUPAN

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan mengenai struktur data dan algoritma menggunakan JavaScript, dapat disimpulkan bahwa pemahaman terhadap function, arrow function, callback, serta array dan method array merupakan dasar yang sangat penting dalam pemrograman.

Function digunakan untuk mengelompokkan kode program sehingga lebih terstruktur dan dapat digunakan kembali. Arrow function memberikan cara penulisan fungsi yang lebih ringkas dan modern sehingga mempermudah pengembangan program. Callback function memungkinkan suatu fungsi dipanggil kembali setelah proses tertentu selesai dijalankan sehingga membantu dalam pengaturan alur program.

Selain itu, array digunakan untuk menyimpan sekumpulan data dalam satu variabel sehingga mempermudah pengelolaan data dalam jumlah banyak. Berbagai method array yang tersedia dalam JavaScript juga membantu dalam melakukan manipulasi dan pengolahan data secara lebih efisien.

Dengan memahami konsep-konsep tersebut, programmer dapat membuat program yang lebih terorganisir, mudah dipahami, dan lebih efisien dalam pengolahan data. Oleh karena itu, penguasaan struktur data dan algoritma menjadi salah satu kemampuan penting dalam pengembangan perangkat lunak.